Multi-DRM protected HLS and DASH from a shared CMAF source

Table of Contents

- Demo content
- Requirement: CPIX
- Packaging media files as CMAF
- Creating HLS Media and Master Playlists
- Creating DASH client manifest (MPD)
- Testing playback

This tutorial describes how to create the 'holy grail' of packaging: using the same media segments to serve both DASH and HLS, with content protection by all three major DRM systems (FairPlay, PlayReady and Widevine).

The output is a set of encrypted CMAF media tracks, along with HLS and DASH manifests that reference these tracks with all DRM signaling included. The tracks are encrypted with different keys to allow for different levels of DRM protection for HD or 4K content.

This tutorial makes use of functionality that was added in CPIX 2.3 specification, for which support was added in version 1.10.16 and beyond.

Demo content

For this tutorial you will need the Tears of Steel content used for evaluation: Your own Video on Demand demo.

Requirement: CPIX

The use of CPIX is required for this workflow, as it is the only way in which specifying the 'cbcs' encryption scheme for DASH output is supported. See CPIX Document Requirements for more information.

The CPIX document shown below uses keys that work with the publicly available Widevine and PlayReady test servers. As there is no easily available FairPlay test server, this setup uses a simple key server that provides the decryption key in the clear for Safari and iOS clients (technically making the stream SAMPLE-AES instead of FairPlay protected, but the set up process is exactly the same).

To work with below CPIX document, save multi-format-drm.cpix in the same location as your `tears-of-steel` directory with the demo content.

```xml
<?xml version='1.0' encoding='UTF-8'?>
<CPIX xmlns:pskc="urn:ietf:params:xml:ns:keyprov:pskc" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="urn:dashif:org:cpix" xsi:schemaLocation="urn:dashif:org:cpix cpix.xsd">
  <!-- Content Keys are listed with their encryption scheme set to "cbcs", and each with explicit IVs to support FairPlay -->
  <ContentKeyList>
    <ContentKey kid="3f0f9c37-bc82-5742-a780-71caaad6d23b" commonEncryptionScheme="cbcs" explicitIV="Pw+cN7yCV0KngHHKqtbSOw==">
      <Data>
        <pskc:Secret>
          <pskc:PlainValue>L0pOSrzvkYW/KUnCq9qPZw==</pskc:PlainValue>
        </pskc:Secret>
      </Data>
    </ContentKey>
    <ContentKey kid="4e2d509a-753f-5e26-b253-cb7d21c3bf05" commonEncryptionScheme="cbcs" explicitIV="Ti1QmnU/XiayU8t9IcO/BQ==">
      <Data>
        <pskc:Secret>
          <pskc:PlainValue>R5Q8JaTx5kARhP+cf9TYBg==</pskc:PlainValue>
        </pskc:Secret>
      </Data>
    </ContentKey>
    <ContentKey kid="80964b5a-22dc-5c93-b18d-8c68b9fb8fc0" commonEncryptionScheme="cbcs" explicitIV="gJZLWiLcXJOxjYxoufuPwA==">
      <Data>
        <pskc:Secret>
          <pskc:PlainValue>IUCA6yDhHCkv2mD7AV+O3Q==</pskc:PlainValue>
        </pskc:Secret>
      </Data>
    </ContentKey>
    <ContentKey kid="6653b449-e474-5b74-9973-1d8e15fdf0d2" commonEncryptionScheme="cbcs" explicitIV="ZlO0SeR0W3SZcx2OFf3w0g==">
      <Data>
        <pskc:Secret>
          <pskc:PlainValue>WIvnfHWl+E3Fi64HSvUVmg==</pskc:PlainValue>
        </pskc:Secret>
      </Data>
    </ContentKey>
    <ContentKey kid="47ed9111-3ef1-5511-92fc-54165fc40f91" commonEncryptionScheme="cbcs" explicitIV="R+2RET7xVRGS/FQWX8QPkQ==">
      <Data>
        <pskc:Secret>
          <pskc:PlainValue>Ilcitl7TYN4d2cPydXiqdQ==</pskc:PlainValue>
        </pskc:Secret>
      </Data>
    </ContentKey>
  </ContentKeyList>
  <DRMSystemList>
    <!-- Widevine DRM with PSSH and empty ContentProtectionData and HLSSignalingData elements, to trigger Packager to generate it -->
    <DRMSystem kid="3f0f9c37-bc82-5742-a780-71caaad6d23b" systemId="edef8ba9-79d6-4ace-a3c8-27dcd51d21ed">
      <PSSH>AAAAMnBzc2gAAAAA7e+LqXnWSs6jyCfc1R0h7QAAABIiCmhvbHlfZ3JhaWxI49yVmwY=</PSSH>
      <ContentProtectionData></ContentProtectionData>
      <HLSSignalingData playlist="media"></HLSSignalingData>
      <HLSSignalingData playlist="master"></HLSSignalingData>
    </DRMSystem>
    <!-- PlayReady DRM with PSSH and empty ContentProtectionData and HLSSignalingData
```

```
elements, to trigger Packager to generate it -->
      <DRMSystem kid="3f0f9c37-bc82-5742-a780-71caaad6d23b" systemId="9a04f079-9840-4286-ab92-e65be0885f95">
<PSSH>AAAHUHBzc2gBAAAAmgTweZhAQoarkuZb4IhflQAAAAU/D5w3vIJXQqeAccqq1tI7Ti1QmnU/XiayU8t9IcO/BYCWS1oi3F
        <ContentProtectionData></ContentProtectionData>
        <HLSSignalingData playlist="media"></HLSSignalingData>
        <HLSSignalingData playlist="master"></HLSSignalingData>
      </DRMSystem>
      <!-- SAMPLE-AES protection with HLSSignalingData containing the base64 encoded #EXT-X-KEY tag, as Packager cannot generate it -->
      <DRMSystem kid="3f0f9c37-bc82-5742-a780-71caaad6d23b" systemId="94ce86fb-07ff-4f43-adb8-93d2fa968ca2">
        <HLSSignalingData playlist="media">I0VYVC1YLUtFWTpNRVRIT0Q9U0FNUExFLUFFUyxVUkk9aHR0cHM6Ly9rZXlzZXJ2ZXIudW5pZmllZC1zdHJ</HLSSignalingData>
        <HLSSignalingData playlist="master">I0VYVC1YLVNFU1NJT04tS0VZOk1FVEhPRD1TQU1QTEUtQUVTLFVSST1odHRwczovL2tleXNlcnZlci51bm</HLSSignalingData>
      </DRMSystem>
      <DRMSystem kid="4e2d509a-753f-5e26-b253-cb7d21c3bf05" systemId="edef8ba9-79d6-4ace-a3c8-27dcd51d21ed">
        <PSSH>AAAAMnBzc2gAAAAA7e+LqXnWSs6jyCfc1R0h7QAAABIiCmhvbHlfZ3JhaWxI49yVmwY=</PSSH>
        <ContentProtectionData></ContentProtectionData>
        <HLSSignalingData playlist="media"></HLSSignalingData>
        <HLSSignalingData playlist="master"></HLSSignalingData>
      </DRMSystem>
      <DRMSystem kid="4e2d509a-753f-5e26-b253-cb7d21c3bf05" systemId="9a04f079-9840-4286-ab92-e65be0885f95">
<PSSH>AAAHUHBzc2gBAAAAmgTweZhAQoarkuZb4IhflQAAAAU/D5w3vIJXQqeAccqq1tI7Ti1QmnU/XiayU8t9IcO/BYCWS1oi3F
        <ContentProtectionData></ContentProtectionData>
        <HLSSignalingData playlist="media"></HLSSignalingData>
        <HLSSignalingData playlist="master"></HLSSignalingData>
      </DRMSystem>
      <DRMSystem kid="4e2d509a-753f-5e26-b253-cb7d21c3bf05" systemId="94ce86fb-07ff-4f43-adb8-93d2fa968ca2">
        <HLSSignalingData playlist="media">I0VYVC1YLUtFWTpNRVRIT0Q9U0FNUExFLUFFUyxVUkk9aHR0cHM6Ly9rZXlzZXJ2ZXIudW5pZmllZC1zdHJ</HLSSignalingData>
        <HLSSignalingData playlist="master">I0VYVC1YLVNFU1NJT04tS0VZOk1FVEhPRD1TQU1QTEUtQUVTLFVSST1odHRwczovL2tleXNlcnZlci51bm</HLSSignalingData>
      </DRMSystem>
      <DRMSystem kid="80964b5a-22dc-5c93-b18d-8c68b9fb8fc0" systemId="edef8ba9-79d6-4ace-a3c8-27dcd51d21ed">
        <PSSH>AAAAMnBzc2gAAAAA7e+LqXnWSs6jyCfc1R0h7QAAABIiCmhvbHlfZ3JhaWxI49yVmwY=</PSSH>
        <ContentProtectionData></ContentProtectionData>
        <HLSSignalingData playlist="media"></HLSSignalingData>
        <HLSSignalingData playlist="master"></HLSSignalingData>
      </DRMSystem>
      <DRMSystem kid="80964b5a-22dc-5c93-b18d-8c68b9fb8fc0" systemId="9a04f079-9840-4286-ab92-e65be0885f95">
<PSSH>AAAHUHBzc2gBAAAAmgTweZhAQoarkuZb4IhflQAAAAU/D5w3vIJXQqeAccqq1tI7Ti1QmnU/XiayU8t9IcO/BYCWS1oi3F
        <ContentProtectionData></ContentProtectionData>
        <HLSSignalingData playlist="media"></HLSSignalingData>
        <HLSSignalingData playlist="master"></HLSSignalingData>
      </DRMSystem>
      <DRMSystem kid="80964b5a-22dc-5c93-b18d-8c68b9fb8fc0" systemId="94ce86fb-07ff-4f43-
```

```
adb8-93d2fa968ca2">
 90         <HLSSignalingData
playlist="media">I0VYVC1YLUtFWTpNRVRIT0Q9U0FNUExFLUFFUyxVUkk9aHR0cHM6Ly9rZXlzZXJ2ZXIudW5pZmllZC1zdHJ
</HLSSignalingData>
 91         <HLSSignalingData
playlist="master">I0VYVC1YLVNFU1NJT04tS0VZOk1FVEhPRD1TQU1QTEUtQUVTLFVSST1odHRwczovL2tleXNlcnZlci51bm
</HLSSignalingData>
 92       </DRMSystem>
 93       <DRMSystem kid="6653b449-e474-5b74-9973-1d8e15fdf0d2" systemId="edef8ba9-79d6-4ace-
a3c8-27dcd51d21ed">
 94         <PSSH>AAAAMnBzc2gAAAAA7e+LqXnWSs6jyCfc1R0h7QAAABIiCmhvbHlfZ3JhaWxI49yVmwY=</PSSH>
 95         <ContentProtectionData></ContentProtectionData>
 96         <HLSSignalingData playlist="media"></HLSSignalingData>
 97         <HLSSignalingData playlist="master"></HLSSignalingData>
 98       </DRMSystem>
 99       <DRMSystem kid="6653b449-e474-5b74-9973-1d8e15fdf0d2" systemId="9a04f079-9840-4286-
ab92-e65be0885f95">
100
<PSSH>AAAHUHBzc2gBAAAAmgTweZhAQoarkuZb4IhflQAAAAU/D5w3vIJXQqeAccqq1tI7Ti1QmnU/XiayU8t9IcO/BYCWS1oi3F
101         <ContentProtectionData></ContentProtectionData>
102         <HLSSignalingData playlist="media"></HLSSignalingData>
103         <HLSSignalingData playlist="master"></HLSSignalingData>
104       </DRMSystem>
105       <DRMSystem kid="6653b449-e474-5b74-9973-1d8e15fdf0d2" systemId="94ce86fb-07ff-4f43-
adb8-93d2fa968ca2">
106         <HLSSignalingData
playlist="media">I0VYVC1YLUtFWTpNRVRIT0Q9U0FNUExFLUFFUyxVUkk9aHR0cHM6Ly9rZXlzZXJ2ZXIudW5pZmllZC1zdHJ
</HLSSignalingData>
107         <HLSSignalingData
playlist="master">I0VYVC1YLVNFU1NJT04tS0VZOk1FVEhPRD1TQU1QTEUtQUVTLFVSST1odHRwczovL2tleXNlcnZlci51bm
</HLSSignalingData>
108       </DRMSystem>
109       <DRMSystem kid="47ed9111-3ef1-5511-92fc-54165fc40f91" systemId="edef8ba9-79d6-4ace-
a3c8-27dcd51d21ed">
110         <PSSH>AAAAMnBzc2gAAAAA7e+LqXnWSs6jyCfc1R0h7QAAABIiCmhvbHlfZ3JhaWxI49yVmwY=</PSSH>
111         <ContentProtectionData></ContentProtectionData>
112         <HLSSignalingData playlist="media"></HLSSignalingData>
113         <HLSSignalingData playlist="master"></HLSSignalingData>
114       </DRMSystem>
115       <DRMSystem kid="47ed9111-3ef1-5511-92fc-54165fc40f91" systemId="9a04f079-9840-4286-
ab92-e65be0885f95">
116
<PSSH>AAAHUHBzc2gBAAAAmgTweZhAQoarkuZb4IhflQAAAAU/D5w3vIJXQqeAccqq1tI7Ti1QmnU/XiayU8t9IcO/BYCWS1oi3F
117         <ContentProtectionData></ContentProtectionData>
118         <HLSSignalingData playlist="media"></HLSSignalingData>
119         <HLSSignalingData playlist="master"></HLSSignalingData>
120       </DRMSystem>
121       <DRMSystem kid="47ed9111-3ef1-5511-92fc-54165fc40f91" systemId="94ce86fb-07ff-4f43-
adb8-93d2fa968ca2">
122         <HLSSignalingData
playlist="media">I0VYVC1YLUtFWTpNRVRIT0Q9U0FNUExFLUFFUyxVUkk9aHR0cHM6Ly9rZXlzZXJ2ZXIudW5pZmllZC1zdHJ
</HLSSignalingData>
123         <HLSSignalingData
playlist="master">I0VYVC1YLVNFU1NJT04tS0VZOk1FVEhPRD1TQU1QTEUtQUVTLFVSST1odHRwczovL2tleXNlcnZlci51bm
</HLSSignalingData>
124       </DRMSystem>
125     </DRMSystemList>
126     <!-- Rules mapping the different keys to different tracks -->
127     <ContentKeyUsageRuleList>
128       <ContentKeyUsageRule kid="3f0f9c37-bc82-5742-a780-71caaad6d23b">
```

```
      <VideoFilter maxPixels="442368"/>
    </ContentKeyUsageRule>
    <ContentKeyUsageRule kid="4e2d509a-753f-5e26-b253-cb7d21c3bf05">
      <VideoFilter minPixels="442369" maxPixels="2073600"/>
    </ContentKeyUsageRule>
    <ContentKeyUsageRule kid="80964b5a-22dc-5c93-b18d-8c68b9fb8fc0">
      <VideoFilter minPixels="2073601" maxPixels="8847360"/>
    </ContentKeyUsageRule>
    <ContentKeyUsageRule kid="6653b449-e474-5b74-9973-1d8e15fdf0d2">
      <VideoFilter minPixels="8847361"/>
    </ContentKeyUsageRule>
    <ContentKeyUsageRule kid="47ed9111-3ef1-5511-92fc-54165fc40f91">
      <AudioFilter/>
    </ContentKeyUsageRule>
  </ContentKeyUsageRuleList>
</CPIX>
```

Packaging media files as CMAF

With the CPIX document and your demo content in place, the first step is to encrypt and package all of the media files as CMAF.

To do this, specify the CPIX document each time you package a media track as CMAF. This will make `mp4split` read the CPIX document and use the appropriate key to encrypt each track.

As the content has a frame rate of 24fps and a GOP length of 96 frames, you should specify this as the fragment duration (or choose a multiple of it):

```bash
#!/bin/bash

# audio
mp4split -o tears-of-steel-aac-64k.cmfa \
  --fragment_duration 96/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel/tears-of-steel-aac-64k.mp4

# video
mp4split -o tears-of-steel-avc1-400k.cmfv \
  --fragment_duration 96/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel/tears-of-steel-avc1-400k.mp4

mp4split -o tears-of-steel-avc1-750k.cmfv \
  --fragment_duration 96/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel/tears-of-steel-avc1-750k.mp4

mp4split -o tears-of-steel-avc1-1000k.cmfv \
  --fragment_duration 96/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel/tears-of-steel-avc1-1000k.mp4

mp4split -o tears-of-steel-avc1-1500k.cmfv \
  --fragment_duration 96/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel/tears-of-steel-avc1-1500k.mp4
```

Creating HLS Media and Master Playlists

For each of the CMAF tracks you need to create an HLS Media Playlist, then from those you can create the Master Playlist.

Media Playlists

Make sure that you specify the CPIX document each time you create an (audio or video) Media Playlist.

Because a 6 seconds segment duration is recommended for HLS, you may want to use the `--fragment_duration` option when creating the Media Playlists, too. Note that in such a case the length that you specify for your HLS segments should be a multiple of the fragment duration of your CMAF source (in this particular case, 8 seconds):

```bash
#!/bin/bash

mp4split -o tears-of-steel-avc1-400k.m3u8 \
  --fragment_duration 192/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel-avc1-400k.cmfv

mp4split -o tears-of-steel-avc1-750k.m3u8 \
  --fragment_duration 192/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel-avc1-750k.cmfv

mp4split -o tears-of-steel-avc1-1000k.m3u8 \
  --fragment_duration 192/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel-avc1-1000k.cmfv

mp4split -o tears-of-steel-avc1-1500k.m3u8 \
  --fragment_duration 192/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel-avc1-1500k.cmfv

mp4split -o tears-of-steel-aac-64k.m3u8 \
  --fragment_duration 192/24 \
  --cpix multi-format-drm.cpix \
  tears-of-steel-aac-64k.cmfa
```

Master Playlist

Specifying the CPIX document or a fragment duration when creating the Master Playlist is not necessary. The command-line is very straightforward:

```bash
#!/bin/bash

mp4split -o master.m3u8 \
  tears-of-steel-avc1-400k.m3u8 \
  tears-of-steel-avc1-750k.m3u8 \
  tears-of-steel-avc1-1000k.m3u8 \
  tears-of-steel-avc1-1500k.m3u8 \
  tears-of-steel-aac-64k.m3u8
```

Creating DASH client manifest (MPD)

For DASH only a single client manifest is required. For this you can enable the `--mpd.inline_drm` option to make sure that explicit DRM information is included in the MPD:

```bash
#!/bin/bash

mp4split -o dash-cbcs.mpd \
  --cpix multi-format-drm.cpix \
  --mpd.inline_drm \
  tears-of-steel-aac-64k.cmfa \
  tears-of-steel-avc1-400k.cmfv \
  tears-of-steel-avc1-750k.cmfv \
  tears-of-steel-avc1-1000k.cmfv \
  tears-of-steel-avc1-1500k.cmfv
```

Testing playback

As we used keys obtained from the Widevine test server we can use the test license server to get a decryption license.

To test playback just upload the files to an https webserver, as this is a requirement for most DRM systems.

As an example, we have added the packaged assets to our demo site:

- HLS
- DASH

The SAMPLE-AES encrypted HLS can be tested in the Safari or iOS native players.

Widevine DRM can be tested in Chrome or Firefox:

- DASH.js reference player (the license server URL must be set manually)
- Shaka DASH
- Shaka HLS